

Welcome to

Web Attacks I – Attacks

Saint Cyber – Cyber Akademiet, 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
academy-2026-day2-web-attacks-I.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



- Identify and explain common web-application weaknesses
- SQL injection, cross-site scripting, cross-site request forgery
- Session handling, file inclusion and security misconfiguration
- Relate them to the OWASP Top 10

Plan for today



- Why web hacking; the OWASP Top 10 as a map
- Injection: SQL injection
- Client-side: XSS and CSRF
- Sessions, TLS/Heartbleed and web scanning

Time schedule



- 08:30–10:00 Web pentest intro; OWASP Top 10; SQL injection
- 10:00–10:15 Break
- 10:15–11:45 XSS, CSRF, sessions, TLS/Heartbleed
- 11:45–12:30 **Exercise** (booklet)
- 12:30–13:15 Lunch

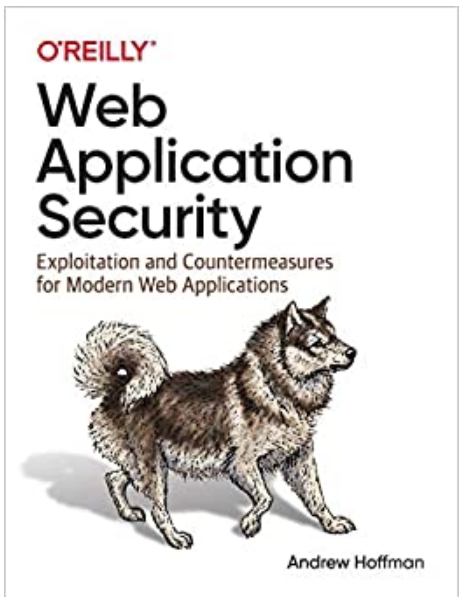
Why talk about hacking web



- Everything is web today
- HTTPS/TLS used for data transport in most applications
- Web APIs are used across the industry
- Even *desktop* and *mobile* application are rarely more than a specialized browser like electronjs.org – Chromium and Node.js
- Also software security and web security has a HUGE overlap

Photo by Pawel Janiak on Unsplash

Web Application Security



Web Application Security, Andrew Hoffman, 2020, ISBN: 9781492053118 called WAS

Primary HTTP mthods



GET Requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect. (This is also true of some other HTTP methods.)[1] The W3C has published guidance principles on this distinction, saying, "Web application design should be informed by the above principles, but also by the relevant limitations."[13] See safe methods below.

POST Requests that the server accept the entity enclosed in the request as a new subordinate of the web resource identified by the URI. The data POSTed might be, for example, an annotation for existing resources; a message for a bulletin board, newsgroup, mailing list, or comment thread; a block of data that is the result of submitting a web form to a data-handling process; or an item to add to a database.[14]

PUT Requests that the enclosed entity be stored under the supplied URI. If the URI refers to an already existing resource, it is modified; if the URI does not point to an existing resource, then the server can create the resource with that URI.[15]

Source: https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

OWASP Top Ten





> The OWASP Top Ten provides a minimum standard for web application security. The OWASP Top Ten represents a broad consensus about what the most critical web application security flaws are.

The Open Web Application Security Project (OWASP)

OWASP produces lists of the most common types of errors in web applications

http://www.owasp.org

Create Secure Software Development Lifecycle

SQL injection



SQL Injection FAQ from Chip Andrews 2001 http://www.sqlsecurity.com:

```
Set myRecordset = myConnection.execute
("SELECT * FROM myTable
WHERE someText ='" & request.form("inputdata") & "'")
```

Using input `exec master..xp_cmdshell 'net user test testpass /ADD' --`

Will perform

```
SELECT * FROM myTable
WHERE someText ='' exec master..xp_cmdshell 'net user test testpass /ADD'--
```

Depending on the access rights this might have full authorization in the database!

Sqlmap



sqlmap is an open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers. It comes with a powerful detection engine, many niche features for the ultimate penetration tester and a broad range of switches lasting from database fingerprinting, over data fetching from the database, to accessing the underlying file system and executing commands on the operating system via out-of-band connections.

Automatic SQL injection and database takeover tool http://sqlmap.org/

sqlmap features



; Features();--

- Full support for **MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, IBM DB2, SQLite, Firebird, Sybase, SAP MaxDB and HSQLDB** database management systems.
- Full support for six SQL injection techniques: **boolean-based blind, time-based blind, error-based, UNION query-based, stacked queries and out-of-band**.
- Support to **directly connect to the database** without passing via a SQL injection, by providing DBMS credentials, IP address, port and database name.
- Support to enumerate **users, password hashes, privileges, roles, databases, tables and columns**.
- Automatic recognition of password hash formats and support for **cracking them using a dictionary-based attack**.
- Support to **dump database tables** entirely, a range of entries or specific columns as per user's choice. The user can also choose to dump only a range of characters from each column's entry.

Not a complete list!

Source: http://sqlmap.org/

10. Cross-Site Scripting (XSS)



> Cross-Site Scripting (XSS) vulnerabilities are some of the most common vulnerabili- ties throughout the internet, and have appeared as a direct response to the increasing amount of user interaction in today's web applications. At its core, an XSS attack functions by taking advantage of the fact that web applica- tions execute scripts on users' browsers. Any type of dynamically created script that is executed puts a web application at risk if the script being executed can be contaminated or modified in any way—in particular by an end user.

Source: *Web Application Security*, Andrew Hoffman, 2020, ISBN: 9781492053118

XSS attacks are categorized a number of ways, with the big three being:

- Stored (the code is stored on a database prior to execution)
- Reflected (the code is not stored in a database, but reflected by a server)
- DOM-based (code is both stored and executed in the browser)

Note: attacks the user, his/her data mostly.

Cross-site scripting



When logged into a site, we have a session identifier – being presented with each HTTP request

If an attacker can *activate* javascript that access and re-send this to another site we have a cross-site scripting attack

We often check using some parameter with code, like this:

```
<A HREF="http://example.com/comment.cgi?
mycomment=<SCRIPT>malicious code</SCRIPT>
">Click here</A>
```

If this code is returned, and active, as part of the returned page and HTML we have a cross-site scripting possibility

This code is executed in the user browser, with the same permissions as the user

11. Cross-Site Request Forgery (CSRF)



Sometimes we already know an API endpoint exists that would allow us to perform an operation we wish to perform, but we do not have access to that endpoint because it requires privileged access (e.g., an admin account).

In this chapter, we will be building Cross-Site Request Forgery (CSRF) exploits that result in an admin or privileged account performing an operation on our behalf rather than using a JavaScript code snippet.

CSRF attacks take advantage of the way browsers operate and the trust relationship between a website and the browser. By finding API calls that rely on this relationship to ensure security—but yield too much trust to the browser—we can craft links and forms that with a little bit of effort can cause a user to make requests on his or her own behalf—unknown to the user generating the request.

- Often seen in small CPE routers in homes, if the user activates and evil link, their router might be reconfigured or taken over

Session IDs



- Session IDs tie the user with the state on the server
- Must be randomly assigned, otherwise an attacker can guess a valid ID
- Common problems, time based or predictable in some way
- Check code for generating IDs or measure - Phase Space Analysis



`https://customerdomain.example.com/api/customer/v1/business/$ENDPPOINT`
ENDPOINT being something like: employeedocuments, clientcontacts, businessPropertyObjects, activities

- We love REST and the world does too

Generic Network Fault Injection



Inserting proxies can allow modification of data in transit

Can be used for random bit corruption

Can often reproduce the data

Automate gathering of evidence

You can use a simple Random TCP/UDP fault injector, but usually you would get better results by conforming to some structure

Various test cases must tried with potential bad data, examples:

- loooong input in all fields and input – buffer overflows
- SQL injection – trying database commands as input
- Cross-site scripting – does inserting JavaScript result in this being included in pages returned
- Random bytes - recommend using real fuzzers that understand target protocol
- Metacharacters like null bytes, or `.,:;/\textbackslash`

Transport Layer Security (TLS)



Originally from Netscape Communications Inc.

Secure Sockets Layer (SSL) was adopted by the IETF newer versions are called Transport Layer Security (TLS)

TLS 1.0 based on generalized version of SSL Version 3.0

RFC-2246 The TLS Protocol Version 1.0 from Januar 1999

RFC-3207 SMTP STARTTLS allows the use of TLS with SMTP mail protocols

Today most sites and servers support TLS Server Name Indication (SNI) name based certificates

Heartbleed CVE-2014-0160



The Heartbleed Bug

The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. This weakness allows stealing the information protected, under normal conditions, by the SSL/TLS encryption used to secure the Internet. SSL/TLS provides communication security and privacy over the Internet for applications such as web, email, instant messaging (IM) and some virtual private networks (VPNs).

The Heartbleed bug allows anyone on the Internet to read the memory of the systems protected by the vulnerable versions of the OpenSSL software. This compromises the secret keys used to identify the service providers and to encrypt the traffic, the names and passwords of the users and the actual content. This allows attackers to eavesdrop on communications, steal data directly from the services and users and to impersonate services and users.



Nok det mest kendte SSL/TLS exploit

Source: http://heartbleed.com/

Heartbleed hacking



```
06b0: 2D 63 61 63 68 65 0D 0A 43 61 63 68 65 2D 43 6F  -cache..Cache-Co
06c0: 6E 74 72 6F 6C 3A 20 6E 6F 2D 63 61 63 68 65 0D  ntrol: no-cache.
06d0: 0A 0D 0A 61 63 74 69 6F 6E 3D 67 63 5F 69 6E 73  ...action=gc_ins
06e0: 65 72 74 5F 6F 72 64 65 72 26 62 69 6C 6C 6E 6F  ert_order&billno
06f0: 3D 50 5A 4B 31 31 30 31 26 70 61 79 6D 65 6E 74  =PZK1101&payment
0700: 5F 69 64 3D 31 26 63 61 72 64 5F 6E 75 6D 62 65  _id=1&card_numbe
0710: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX   r=4060xxxx413xxx
0720: 39 36 26 63 61 72 64 5F 65 78 70 5F 6D 6F 6E 74  96&card_exp_mont
0730: 68 3D 30 32 26 63 61 72 64 5F 65 78 70 5F 79 65  h=02&card_exp_ye
0740: 61 72 3D 31 37 26 63 61 72 64 5F 63 76 6E 3D 31  ar=17&card_cvn=1
0750: 30 39 F8 6C 1B E5 72 CA 61 4D 06 4E B3 54 BC DA  09.l..r.aM.N.T..
```

- Obtained using Heartbleed proof of concepts - Gave full credit card details
- "can XXX be exploited- yes, clearly! PoCs ARE needed
  without PoCs even Akamai wouldn't have repaired completely!
- The internet was ALMOST fooled into thinking getting private keys from Heartbleed was not possible - scary indeed.

sslscan



```
root@kali:~# sslscan --ssl2 web.kramse.dk
Version: 1.10.5-static
OpenSSL 1.0.2e-dev xx XXX xxxx

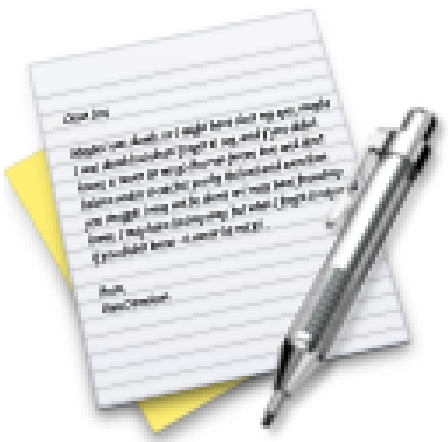
Testing SSL server web.kramse.dk on port 443
...
  SSL Certificate:
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength:    2048

Subject:  *.kramse.dk
Altnames: DNS:*.kramse.dk, DNS:kramse.dk
Issuer:   AlphaSSL CA - SHA256 - G2
```

Source: Originally sslscan from http://www.titania.co.uk but use the version on Kali

SSLscan can check your own sites, while Qualys SSLLabs only can test from hostname

Exercise

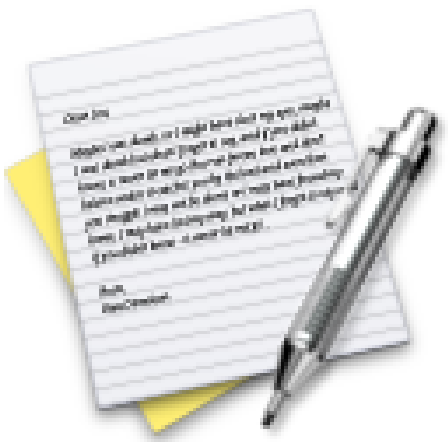


Now lets do the exercise

??

which is number **??** in the exercise PDF.

Exercise



Now lets do the exercise

??

which is number **??** in the exercise PDF.

Scanning Web Applications / Web Servers



Description Nikto is an Open Source (GPL) web server scanner which performs comprehensive tests against web servers for multiple items, including over 6700 potentially dangerous files/programs, checks for outdated versions of over 1250 servers, and version specific problems on over 270 servers. It also checks for server configuration items such as the presence of multiple index files, HTTP server options, and will attempt to identify installed web servers and software. Scan items and plugins are frequently updated and can be automatically updated.

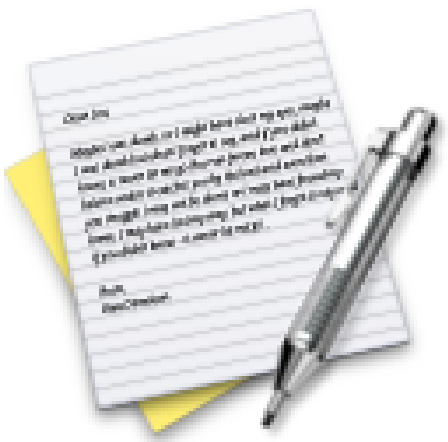
Source: https://cirt.net/Nikto2

- Nikto is quite old, some would say mature tool. Always recommended to run this, cheap, quick, finds interesting links
- Whatweb can be used to check up on a site, what framework does it use, how is it configured, where is it hosted

- Then afterwards I recommend Burp Suite professsional https://portswigger.net/burp/pro
- The exercise booklet also has a few extra exercises about session tokens, and SQL injection



Exercise

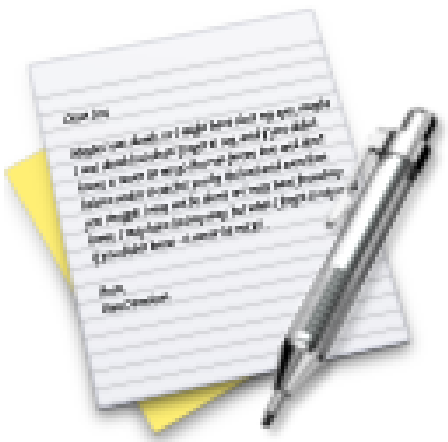


Now lets do the exercise

??

which is number **??** in the exercise PDF.

Exercise



Now lets do the exercise

??

which is number **??** in the exercise PDF.

Questions?



Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hlk@zencurity.com

File inclusion (LFI / RFI)



- **FIXME:** not in these decks – add local/remote file inclusion (obj 7)

Security misconfiguration



- **FIXME:** defaults, verbose errors, missing security headers (obj 7)

Summary



- Main web weaknesses: injection, XSS, CSRF, sessions, file inclusion, misconfiguration
- Next: tools, labs and defences